SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code.* *(BL3)*

Assessment Tool Weight-age: 40%

QUESTIONS: 5 **Total Marks:50**

Annexure A

SIMULATION TASK

Code of Conduct:

I Ramya S S/192421226 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Ramya S S

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts

Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

SIMULATION TASK

SIMULATION TASK – COMPILER DESIGN

Aim:

To simulate compiler optimization techniques, basic block formation, DAG construction, register allocation, and target code generation.

1. Peephole Optimization

Given Code
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y

Step 1: Algebraic Simplification
Using algebraic identities:
x * 1 = x
t2 - 0 = t2
Therefore:
t1 = x
t2 = t1 + y
t3 = t2
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y

Step 2: Constant Folding
Since:
t3 * 0 = 0
we get:
t4 = 0
if 0 != 0 goto L1
The condition is always false, so the jump is removed.

Step 3: Dead Code Elimination
t1, t2, t3, and t4 do not contribute to the final useful computation. Hence they are removed.
Optimized Code
t5 = y + y
Optimizations Used

Optimization	Example
Algebraic simplification	x*1 → x, x-0 → x

Constant folding	**t3*0 → 0**
Dead code elimination	**Removes unused temporaries**
Constant condition	**if 0!=0 removed**

---

2. Basic Blocks and Control Flow Graph

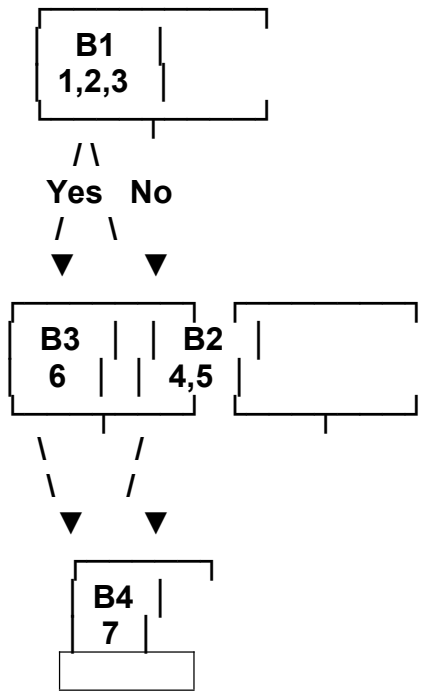
Given Code
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2

Rules for Leaders
A statement is a leader when:

1. **It is the first statement.**
2. **It is the target of a jump.**
3. **It immediately follows a jump statement.**

Leaders
1, 4, 6, 7
Basic Blocks
B1
1. x = a + b
2. y = x * c
3. if y > 50 goto 6
B2
4. z = y - d
5. goto 7
B3
6. z = y + d
B4
7. w = z * 2

Control Flow Graph



CFG Edges
B1 → B3 (y > 50)
B1 → B2 (y ≤ 50)
B2 → B4
B3 → B4

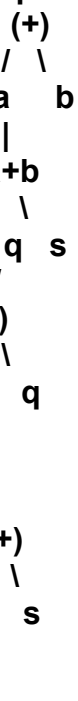
---

3. DAG and Common Sub-Expression

Given Basic Block
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s

Common Sub-Expression
The expression:
a + b
occurs three times:
p = a + b
q = a + b
s = a + b
Therefore, it should be calculated only once.

DAG Representation



Optimized Intermediate Code

```
p = a + b
q = p
r = p * q
s = p
t = r + s
```

Target Code Using Two Registers

```
LOAD R0, a
ADD  R0, b
MOV  R1, R0
MUL  R0, R1
ADD  R0, R1
STORE t, R0
```

Register Allocation

Step	R0	R1
Load a	a	—
Compute a+b	a+b	—
Save common value	a+b	a+b
Multiply	$(a+b)^2$	a+b
Final addition	$(a+b)^2+(a+b)$	a+b

Thus, the repeated a+b calculation is eliminated.

4. Code Generation with Two Registers

Given Code
t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4

Registers available:
R0, R1
Target Code
LOAD R0, a
ADD R0, b

LOAD R1, c
MUL R1, d

SUB R0, R1

LOAD R1, e
ADD R1, f

DIV R0, R1
STORE t5, R0

Register Descriptor and Address Descriptor

Step	R0	R1	Address Descriptor
Initial	Empty	Empty	Variables in memory
t1=a+b	t1	Empty	t1 → R0
t2=c*d	t1	t2	t1 → R0, t2 → R1
t3=t1-t2	t3	t2	t3 → R0, t2 → R1
t4=e+f	t3	t4	t3 → R0, t4 → R1
t5=t3/t4	t5	t4	t5 → R0, t4 → R1

Explanation

- **R0 stores the result of a+b.**
- **R1 stores c*d.**
- **After subtraction, R0 contains t3.**
- **R1 is reused for e+f.**
- **Finally, R0/R1 produces t5.**

---

5. RISC Target Code Generation
Expression
(a - b) * (c + d) - e
Target Instructions
First calculate (a-b):
LOAD R0, a
LOAD R1, b

SUB R0, R1
Now calculate (c+d):
LOAD R2, c
LOAD R3, d
ADD R2, R3
Multiply the two results:
MUL R0, R2
Subtract e:
LOAD R1, e
SUB R0, R1
Store the final result:
STORE result, R0
Complete Code
LOAD R0, a
LOAD R1, b
SUB R0, R1

LOAD R2, c
LOAD R3, d
ADD R2, R3

MUL R0, R2

LOAD R1, e
SUB R0, R1

STORE result, R0

Additional Given Code – Peephole Optimization

Given
t1 = a + b
t2 = t1 + 0
t3 = t2 * 1
if t3 == 0 goto L1
t4 = t3 + t3
Using:
x + 0 = x
x * 1 = x
we get:
t1 = a + b
t2 = t1
t3 = t2
if t3 == 0 goto L1
t4 = t3 + t3

RUBRICS FOR CALCULATION

Simulation tools

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation